Datenbank-Zugriffe

Datenbankprogrammierung

Inhaltsverzeichnis

- Definition und Arten von Zugriffen
- CRUD-Operations
- Datenbank Management Systemen
- Transaktionen und ACID-Eigenschaften
- Programmiersprachen (+ Frameworks)
- Prepared Statements
- Stored Procedure, Views, Indizies
- Trigger und Schedules
- REST-API
- Sicherheitsmechanismen

Arten von Datenbank-Zugriffen

Definition und Überblick

- Definition: Interaktion mit einer Datenbank zur Speicherung, Abfrage und Manipulation von Daten.
- Direkte Verbindung (z.B. über DBMS), Datenbanktreiber (z.B. JDBC), ORM-Frameworks usw.

Was wollen wir mit dem Zugriff erreichen?

CRUD-Operations

Definition

- **C**reate -> Datensatz anlegen
- **R**ead -> Datensatz lesen
- **U**pdate -> Datensatz aktualisieren
- **D**elete -> Datensatz löschen

CRUD-Operations

SQL Beispiele

- INSERT INTO Kunde (ID, Name) VALUES (1, 'Max Mustermann');
- UPDATE Kunde SET Alter = 31 WHERE ID = 1;
- SELECT * FROM Kunde;
- DELETE FROM Kunde WHERE ID = 1;

Datenbank Management Systeme

Überblick

- Softwareanwendungen um die Verwaltung von Datenbanken zu erleichtern.
- Bieten Mechanismen zur Organisation, Speicherung, Abfrage und Aktualisierung von Daten.
- Beispiele: PostgreSQL, MySQL, MariaDB, usw.

pgAdmin File Object Tools Help

Browser

- Servers (1)
  - PostgreSQL 14
    - Databases (7)
    - Login/Group Roles
    - Tablespaces

MiguelTroyano/postgres@PostgreSQL 14 * Properties SQL

Query Editor Query History

```
--MiguelTroyano.com
insert into empleados (nombre, puesto, sueldo)
values ('Miguel Troyano','Consultor', 60000);
```

Data Output Explain Messages Notifications

```
INSERT 0 1

Query returned successfully in 144 msec.
```

https://www.migueltroyano.com/wp-content/uploads/2022/01/postgresql_insert_into_un_valor.jpg

Transaktionen und ACID Eigenschaften

Überblick

- **A**tomicity: Alle Operationen einer Transaktion werden entweder vollständig ausgeführt oder gar nicht.
- **C**onsistency: Die Datenbank bleibt in einem konsistenten Zustand, unabhängig von Transaktionsfehlern.
- **I**solation: Transaktionen werden voneinander isoliert ausgeführt, um Konflikte zu vermeiden.
- **D**urability: Nach dem erfolgreichen Abschluss einer Transaktion bleiben die Änderungen dauerhaft in der Datenbank gespeichert.

Programmiersprachen (+ Frameworks)

Überblick

- Java mit dem JDBC-Framework für den Datenbankzugriff.
- Python mit dem SQLAlchemy-Framework für die Interaktion mit relationalen Datenbanken.
- C# mit ADO.NET für den Datenbankzugriff im .NET-Framework.
- PHP mit dem MySQLi-Modul für den Zugriff auf MySQL-Datenbanken.

Programmiersprachen (+ Frameworks)

Verbindungsaufbau

- Passendes Framework wählen (Sicherstellung der Kompatibilität zum DBMS)
- Importieren erforderlicher Bibliotheken und/oder Module
- Benötigte Informationen zur Verbindung
  - Hostname/IP, Port
  - Anmeldeinformationen (Benutzername, Passwort)
- Anweisungen ausführen

Programmiersprachen (+ Frameworks)

Verbindungsaufbau Beispiel

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseConnectionExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/meineDatenbank";
        String username = "benutzername";
        String password = "passwort";
        try {
            Connection connection = DriverManager.getConnection(url, username, password);
            System.out.println("Verbindung zur Datenbank hergestellt!");
            // Führe hier Datenbankoperationen aus
           connection.close();
            System.out.println("Verbindung zur Datenbank geschlossen!");
        } catch (SQLException e) {
            System.out.println("Fehler beim Verbindungsaufbau zur Datenbank: " + e.getMessage());
        }
    }
}
```

Programmiersprachen (+ Frameworks)

Datenbankoperation Beispiel

```
// Führe hier Datenbankoperationen aus
Statement statement = connection.createStatement()
Scanner scanner = new Scanner(System.in);

System.out.print("Bitte geben Sie den Namen des Kunden ein: ");
String name = scanner.nextLine();

// Beispiel-Abfrage: INSERT
String insertQuery = "INSERT INTO Kunden (Name) VALUES ('" + name + „')";

statement.executeUpdate(insertQuery);
System.out.println("Datensatz eingefügt");
```

Programmiersprachen (+ Frameworks)

Gefahr vor SQL-Injections

```
// Beispiel-Abfrage: INSERT
String insertQuery = "INSERT INTO Kunden (Name) VALUES ('" + name + "')";
```

```
// Beispiel-Abfrage: INSERT
String insertQuery = "INSERT INTO Kunden (Name) VALUES ('John'); DROP TABLE Kunden;--')";
```

Prepared Statements

Definition

- Vorab kompilierte SQL-Abfragen, bei denen Parameter anstelle von Werten verwendet werden.
- Vorteil: Schutz vor SQL-Injektionen, verbesserte Leistung durch wiederverwendbare Abfragen, Trennung von SQL-Code und Daten und Vorabkompilierung für effiziente Ausführung.

Prepared Statements

Datenbankoperation Beispiel

```
// Führe hier Datenbankoperationen aus
Statement statement = connection.createStatement()
Scanner scanner = new Scanner(System.in);

System.out.print("Bitte geben Sie den Namen des Kunden ein: ");
String name = scanner.nextLine();

// Beispiel-Abfrage: INSERT
String insertQuery = "INSERT INTO Kunden (Name) VALUES (?)";
PreparedStatement preparedStatement = connection.prepareStatement(insertQuery);
preparedStatement.setString(1, name);

preparedStatement.executeUpdate(insertQuery);
System.out.println("Datensatz eingefügt");
```

Stored Procedure

Definition

- Vordefinierter SQL-Code für wiederholte Aufgaben oder komplexe Operationen.
- Verbesserte Datenbankleistung und Konsistenz.
- Erhöhte Sicherheit und Zugriffskontrolle.
- Speicherung auf dem Datenbankserver.
- Aufruf von verschiedenen Anwendungen oder Benutzern.
- Flexibilität durch Parameterübergabe.
- Eindeutiger Name für gespeicherten Code.
- Ausführung durch einfachen Aufruf des Namens mit optionalen Parametern.

Stored Procedure

Beispiel

```
// Erstellen des Procedure im DBMS
CREATE PROCEDURE GetCustomerById(
    IN customerId INT
)
BEGIN
    SELECT * FROM Kunden WHERE id = customerId;
END

————————————————————————————————————————————————————————————————————————————————

// Aufruf des Procedure in Java (jdbc)
CallableStatement callableStatement = connection.prepareCall("{CALL GetCustomerById(?)}")) {

callableStatement.setInt(1, 123); // Beispiel-Kunden-ID

ResultSet resultSet = callableStatement.executeQuery();
```

Views

Definition

- Virtuelle Tabellen in einer Datenbank.
- Logische Sicht auf die Daten.
- Erstellung aus vorhandenen Tabellen.
- Vereinfachung des Datenzugriffs und der Datenbankabfragen.
- Steuerung des Datenzugriffs und Ausblenden von Spalten.
- Verbesserung der Datenkonsistenz und -integrität.
- Unterstützung für komplexe Datenabfragen, Joins und Aggregationsfunktionen.
- Optimierung des Datenzugriffs und der Datenverwaltung.
- Verkürzung der Entwicklungszeit und Bereitstellung einer konsistenten Sicht auf die Daten.

Views

Beispiel

```
// Erstellen einer View
CREATE VIEW KundenBestellungen AS
SELECT Kunden.Name, COUNT(Bestellungen.ID) AS AnzahlBestellungen
FROM Kunden
LEFT JOIN Bestellungen ON Kunden.ID = Bestellungen.Kunden_ID
GROUP BY Kunden.ID;

————————————————————————————————————————————————————————————

// Ergebnis
+————————+————————————————+
| Name       |   AnzahlBestellungen |
+————————+————————————————+
| Max Mustermann   |      2          |
| Lisa Schmidt         |       1         |
| Peter Müller         |       2         |
+————————+————————————————+
```

Indizies

Definition

- Beschleunigung des Datenzugriffs
- Bessere Performance bei großen Datenmengen
- Schnelles Suchen, Sortieren und Filtern von Daten
- Datenintegrität durch eindeutige Schlüssel und Wertebeschränkungen
- Beachtung des zusätzlichen Speicherbedarfs und Wartungsaufwands
- Gezielte Erstellung für häufig genutzte Spalten
- Regelmäßige Überprüfung und Wartung erforderlich
- Anwendungsanforderungen als Grundlage für Erstellung

Indizies

Beispiel

```
// Erstellen eines Indizes
CREATE INDEX idx_Kundegeschlecht ON Kunde (Geschlecht);

_____________________________________________________________________

// Aufruf als SQL
SELECT Name FROM Kunde WHERE Geschlecht = 'weiblich';
```

Trigger
Definition

- Automatisierte Reaktionen auf Ereignisse in der Datenbank
- Gewährleistung von Datenkonsistenz und -integrität
- Automatisierung von Abläufen und Durchsetzung von Anforderungen
- Protokollierung von Datenänderungen und Überwachung von Aktivitäten
- Beachtung von Ausführungsreihenfolge und Leistungsauswirkungen
- Anwendungsfälle: Datenvalidierung, Tabellenaktualisierung, Benachrichtigungen
- Effektives Werkzeug für komplexe Geschäftsregeln und Datenintegrität

Trigger

Beispiel

```
// Erstellen eines Trigger
CREATE TRIGGER neuer_kunde
AFTER INSERT ON kunden
FOR EACH ROW
BEGIN
  INSERT INTO kunden_logs (kunden_id, log_nachricht, log_datum)
  VALUES (NEW.kunden_id, 'Neuer Kunde erstellt', NOW());
END;
```

Schedules

Definition

- Automatische Ausführung von Aktionen zu festgelegten Zeitpunkten oder in regelmäßigen Intervallen.
- Effiziente Automatisierung wiederkehrender Aufgaben.
- Zeitgesteuerte Durchführung von Datenbereinigung, Berichterstellung und Aktualisierungen.
- Flexibilität bei der Planung und Durchführung von Aktionen.
- Zeit- und Aufwandsersparnis durch automatisierte Zeitpläne.
- Verbesserte Produktivität durch Automatisierung wiederholter Aufgaben.

Schedules

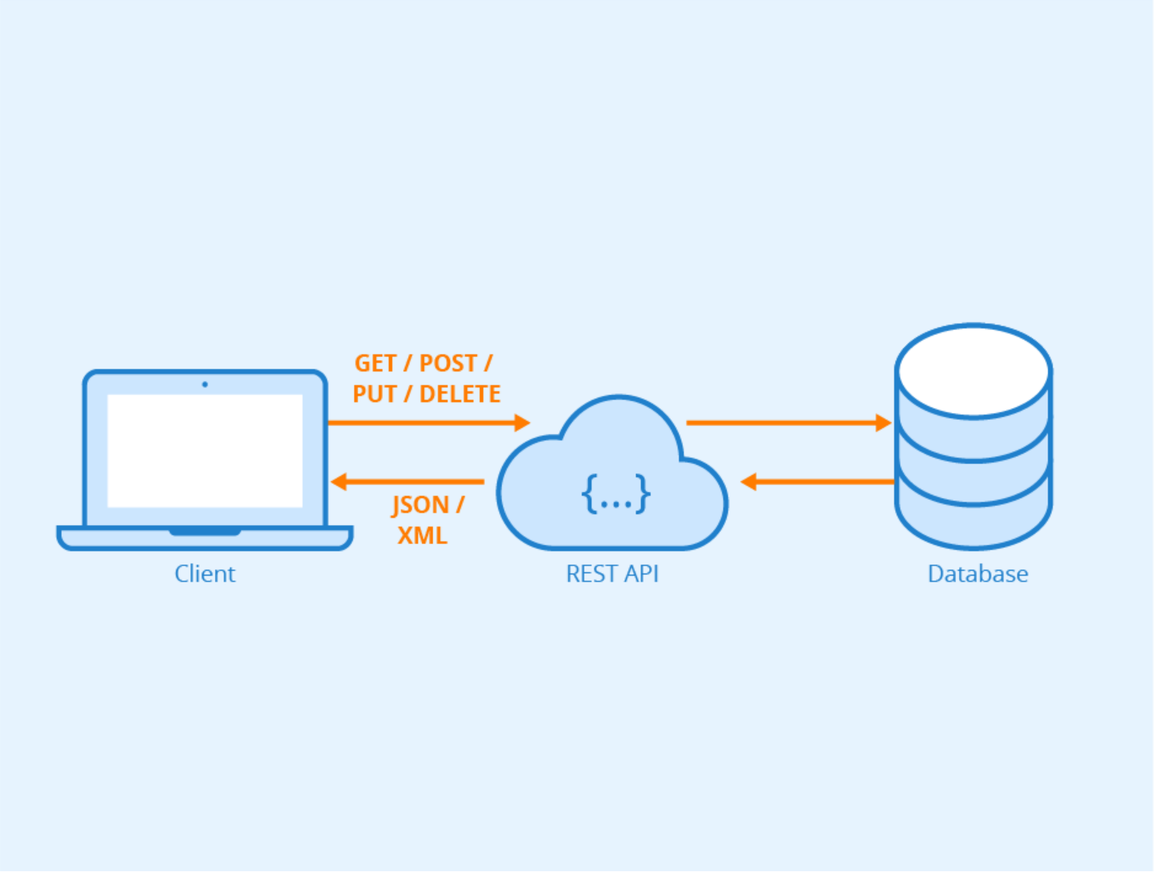
Beispiel

```
// Erstellen eines Schedule
CREATE EVENT NeueKundenEvent
ON SCHEDULE EVERY 1 DAY
STARTS CURRENT_DATE + INTERVAL 9 HOUR
DO
BEGIN
   CALL create_new_customer();
END;
```

REST-API

Gängige Zugriff-Methode

- Standardisierte Kommunikation zwischen Systemen
- Lose Kopplung von Client und Server
- Flexibilität und Skalierbarkeit
- Unterstützung verschiedener Datenformate
- Zugriff über standardisierte Methoden
- Implementierung von Sicherheit und Authentifizierung
- Plattformunabhängigkeit Klare Trennung von Frontend und Backend
- Einfache Wartung und Weiterentwicklung



https://www.astera.com/wp-content/uploads/2020/01/rest.png

Sicherheitsmechanismen

Benutzer- und Rollenverwaltung

- Zuweisung von Benutzerkonten
- Definition von Rollen mit Zugriffsrechten
- Steuerung von Berechtigungen auf Datenbankebene
- Vereinfachte Verwaltung von Zugriffsrechten durch Rollen
- Einschränkung des Zugriffs auf sensible Daten
- Erhöhung der Sicherheit der Datenbank

Sicherheitsmechanismen

Berechtigungen auf Tabellen- und Spaltenebene

- Granulare Steuerung des Datenzugriffs
- Festlegung von Berechtigungen auf Tabellen- und Spaltenebene
- Zuweisung von SELECT, INSERT, UPDATE oder DELETE Berechtigungen
- Einschränkung oder Erlaubnis bestimmter Operationen auf Daten
- Gewährleistung, dass Benutzer nur auf autorisierte Daten zugreifen und sie ändern können
- Sicherung der Datenintegrität
- Schutz vor unbefugtem Zugriff durch die richtige Konfiguration von Berechtigungen

Sicherheitsmechanismen

Verschlüsselung von Daten

- Schutz sensibler Informationen in der Datenbank
- Verwendung von Verschlüsselungsalgorithmen
- Verschlüsselte Speicherung von Daten in der Datenbank
- Zugriff nur mit dem entsprechenden Entschlüsselungsschlüssel
- Unlesbarkeit der Daten bei unautorisiertem Zugriff
- Anwendung der Verschlüsselung auf Spalten- oder Datenbankebene
- Sicherung der Datenbank mit umfassender Verschlüsselung

Sicherheitsmechanismen

Überwachungsmechanismen zur Erkennung verdächtiger Aktivitäten

- Aufzeichnung von Ereignissen wie Anmeldeversuchen, Datenänderungen und Zugriffsversuchen
- Implementierung von Überwachungsregeln
- Regelmäßige Überprüfung von Logs
- Identifizierung ungewöhnlicher Aktivitäten
- Ergreifen angemessener Maßnahmen zur Sicherung der Datenbank

Vielen Dank für Ihre Aufmerksamkeit

Übungen

- Folie 13: Wo liegt der Fehler? (Prepared Statement)
- Folie 22: Welche Spalte in der Tabelle Kunde würde sich zum indizieren eignen?
- Im Anschluss:
  - Welches System ist für das einhalten der ACID-Eigenschaften verantwortlich?
  - Wie präsentiert man SQL Injections?
  - Wann eignet sich das indizieren?
  - Welche Datenbank Zugriffe habe Sie bisher verwendet?